Redis常见场景题

string

redis的string就是一个单个kv的结构.

1. 最常用的就是缓存对象
2. 可以使用setnx实现一个分布式锁, 之前文章细讲过了.
3. 可以实现共享session.
    1. 用户登录生成token, 以token为key, 以用户信息为value, 给一个过期时间, 存到redis中.
    2. 用户一段时间没访问系统, 就过期失效需要重新登录. 只要用户一访问, 那通过拦截器就给token续期.

```
# 思路如下:

# 用户登录后生成token
String token = UUID.randomUUID().toString();

# 以token为key, 以用户信息为value, 给一个过期时间, 存到redis中
setex <token> <过期时间> <用户信息json>

# 获取用户信息
get <token>

# 判断用户是否登录/校验token有效性
exists <token>

# 拦截器中token续期
expire <token> <新过期时间>
```

4. string有原子性的自增操作, 所以可以作为一个计数器, 记录某个网页的访问量, 记录视频的播放次数.
    1. 以网页路径或者视频id为key, 以次数为value. 访问一次, 自增一次.

```redis
# 初始化计数器，如果key不存在，初始化为0，假设视频id为123
set video:play:123 0

# 每次访问/播放时，执行原子自增
incr video:play:123   # 返回值就是自增后的值，如1,2,3

# 获取当前计数值
get video:play:123
```

5. 可以做计数器限流
    1. 比如针对ip/接口做计数器限流.
    2. 以ip或者接口的路径为key, 以访问次数为value, 访问一次, 自增一次. 当到了某个阈值, 比如说每分钟超过120次, 就拒绝请求.
    3. lua脚本示例

```lua
-- KEYS[1] = 限流key (如rate_limit:ip:192.168.1.1)
-- ARGV[1] = 限流时间窗口(秒)
-- ARGV[2] = 最大请求阈值

local current = redis.call('INCR', KEYS[1])

-- 首次设置过期时间
if current == 1 then
    redis.call('EXPIRE', KEYS[1], ARGV[1])
end
```

```lua

-- 检查是否超过阈值
if current > tonumber(ARGV[2]) then
    return 0  -- 超过阈值
end

return 1  -- 允许访问
```

4. 但是这种计数器限流的问题就在于, 第一分钟和第二分钟的交界处, 会允许2倍流量. 比如第60秒来了120个请求, 第61秒, 又来120个请求, 这是拦不住的.

hash

hash结构可以理解为是一个能写入多组kv的string.

1. hash结构可以存对象, 并且可以分开存储对象的不同字段, 如果要精细的修改某个对象的某个属性, 用hash就是比较合适的.
2. hash结构还可以做购物车
    1. 以用户id为key, 商品id为field,商品数量为value

```redis
# 用户id=1001
# 添加商品101，数量为1
HSET cart:1001 101 1

# 再添加商品102，数量为2：
HSET cart:1001 102 2

# 修改商品101的数量为3（直接设置）：
HSET cart:1001 101 3

# 给商品101增加2个（此时变为5个）：
HINCRBY cart:1001 101 2

```

```redis
# 获取商品101的数量
HGET cart:1001 101

# 获取所有商品
HGETALL cart:1001

# 删除商品102
HDEL cart:1001 102

# 清空购物车
DEL cart:1001
```

3. hash结构也可以做一个简单的动态配置管理, 比如说做一些功能开关之类的. 或者做一个动态参数管理.

   1. 以某某config为key, 以功能为feild, 以true/false为value.

```redis
# 开启搜索建议功能
HSET global:config search_suggestions "true"

# 默认使用暗黑模式的ui
HMSET global:config ui_mode "dark"

# 获取搜索建议功能的配置值
HGET global:config search_suggestions

# 关闭搜索建议功能
HSET global:config search_suggestions "false"
```

   2. 这样使用相当于把redis当成了一个简单的配置中心. 但是redis毕竟是基于内存的没有那么可靠, 而且配置发布变更也没有权限管控, 所以还是比较危险. 最好还是nacos或者apollo这种专业的配置中心.

4. 多维度数据统计, 比如说记录不同地区的视频访问量

1. 以视频id为key, 以地区为field, 以访问次数为value. 访问一次自增一次.

```redis
# 初始化或增加特定地区的访问量
# 比如当视频ID为12345的视频被北京用户访问一次
HINCRBY video_views:12345 beijing 1

# 获取ID为12345的视频北京地区的访问量
HGET video_views:12345 beijing

# 获取ID为12345的视频的所有地区访问量
HGETALL video_views:12345
```

list

redis的list其实就是一个双端队列, 有序可重复, 即可以作为一个先进先出的队列去用, 又能作为一个后进先出的栈去用.

1. redis的list可以作为一个简单的消息队列去用.
    1. 消息生产者直接lpush消息到list, 消息消费者使用BRPOP阻塞式的读取数据.

    ```redis
    # 给消息队列插入数据
    LPUSH my_queue "{msg_id:1, type: 123, content: xxxx}"

    # 阻塞式消费队列数据
    BRPOP my_queue
    ```

    2. 要保证消息不被重复处理, 保证消息的幂等性, 就得给消息分配唯一id, 消费者保存自己处理过的消息id, 避免重复消费.

    3. list做一个简单的消息队列还是可以的, 但是没办法去消息持久化, 没办法搞死信队列, 也没办法让多个消费者消费同一条消息. 但是如果说系统比较小, 不想引入那么重的mq, 那么用redis的list做消息队列也是可以满足基本需求的. 如果redis版本比较高, 可以用stream, stream是redis高版本专门做消息队列的工具.
2. list可以做一个简单的实时消息流.
    1. 比如说以用户id为key, 以消息id为value存到list中. 当某个人发帖子, 或者发朋友圈, 就给放入到用户id对应的list中. 当某个人打开朋友圈, 就从list中读取对应的前几条消息.

```redis
# 用户1001发了一条朋友圈， 该朋友圈id=12345， 将其推送给所有好友的列表
# 假设用户1001的好友列表是[2001, 2002, 2003]. 可使用管道（pipeline）来批量操作
LPUSH feed:2001 "12345"
LPUSH feed:2002 "12345"
LPUSH feed:2003 "12345"

# 当用户2001打开朋友圈，获取前10条消息ID
LRANGE feed:2001 0 9
```

set

redis的set结构, 就是一个无序无重复的集合.

1. set可以做抽奖系统
    1. 直接用把奖品add进set集合
    2. 如果是不允许重复中奖的场景. 那就直接SPOP随意移除一个元素, 这个元素就是用户的奖品.

3. 如果是允许重复中奖的场景, 那就直接S RAND MEMBER随机获取一个元素, 这个就是用户的奖品.

```redis
# 添加奖品到奖池
SADD prize_pool "iPhone15" "AirPods" "100元券" "50元券" "10元券" "谢谢参与"

# 查看所有奖品
SMEMBERS prize_pool

# 查看奖品数量
SCARD prize_pool

# 用户抽奖（不允许重复中奖）
SPOP prize_pool

# 用户抽奖（允许重复中奖）
SRANDMEMBER prize_pool
```

2. 因为set无序无重复的特性, 可以做一些数据去重
3. 做查询时, SISMEMBER这个命令时间复杂度O1, 所以可以做一个白名单, 判断某个数据是否在白名单.
4. set还可以做点赞
   1. 以帖子或者朋友圈id为key, 以点赞用户id为value存入set. 谁点赞了, 就放入对应的set集合中.

```redis
# id为1001用户点赞了id为12345的帖子
SADD post:12345:likes 1001

# 取消点赞（从Set中移除用户ID）
SREM post:12345:likes 1001

# 获取所有点赞用户ID
```

```redis
SMEMBERS post:12345:likes

# 获取点赞数量
SCARD post:12345:likes

# 检查用户1001是否对帖子12345点过赞。返回1表示点过赞，0表示没有。
SISMEMBER post:12345:likes 1001
```

5. set还可以做交集, 并集, 差集等. 所以可以做共同好友, 共同关注, 共同兴趣等.
    1. 以用户id为key, 以好友id为value, 存入set. 然后取交集就能得到共同好友.
    2. 也可以存用户兴趣标签, 以用户id为key, 以兴趣标签为value, 存入set. 然后不同用户取交集就能计算不同用户的共同兴趣.

```redis
# 用户1001的兴趣标签
SADD user:1001:tags "篮球" "电影" "编程" "旅行"

# 用户1002的兴趣标签
SADD user:1002:tags "电影" "美食" "摄影" "旅行"

# 获取用户1001的所有标签
SMEMBERS user:1001:tags

# 检查用户是否对某标签感兴趣
SISMEMBER user:1001:tags "编程"

# 获取用户标签数量
SCARD user:1001:tags

# 计算两个用户的共同兴趣
SINTER user:1001:tags user:1002:tags
```

```redis

# 计算用户1001独有的兴趣
SDIFF user:1001:tags user:1002:tags
```

zset

zset的特点是有序无重复. 每个元素都有个分数(score)用于排序. 所以比较适合排序场景.

1. 排序最常用的场景就是排行榜
    1. 比如说, 积分排行榜, 以积分做为score, 用户id为value, 存入zset.
    2. 然后使用`zrange` 就能获取从小到大的top n、 `zrevrange` 就能获取到从大到小的top n

```redis
# 添加用户积分（用户不存在时添加，存在时更新）
ZADD leaderboard 500 1001
ZADD leaderboard 750 1002
ZADD leaderboard 1200 1003
...

# 用户1001增加50积分
ZINCRBY leaderboard 50 1001

# 用户1002减少30积分
ZINCRBY leaderboard -30 1002

# 获取前10名（从高到低）
ZREVRANGE leaderboard 0 9 WITHSCORES

# 获取后10名（从低到高）
ZRANGE leaderboard 0 9 WITHSCORES

```

```redis
# 获取用户排名（从高到低）
ZREVRANK leaderboard 1001

# 获取用户积分
ZSCORE leaderboard 1001

# 获取排行榜总人数
ZCARD leaderboard

# 查询积分在800-1000之间的用户
ZRANGEBYSCORE leaderboard 800 1000 WITHSCORES

# 删除用户
ZREM leaderboard 1001
```

2. 滑动窗口限流
    1. 比如说针对ip限流, 以用户ip为 Key，请求时间戳为 `score`，请求id为value. 存入zset.
    2. 这就能够统计, 一分钟内某ip的请求次数. 然后超过阈值就限流.

```lua
-- KEYS[1]: IP限流键（如ip_requests:192.168.1.1）
-- ARGV[1]: 当前时间戳（毫秒）
-- ARGV[2]: 时间窗口（秒）
-- ARGV[3]: 最大请求数

local current_time = tonumber(ARGV[1])
local window_seconds = tonumber(ARGV[2])
local max_requests = tonumber(ARGV[3])

-- 计算窗口开始时间
local window_start = current_time - window_seconds * 1000

```

```lua
-- 清理过期请求
redis.call('ZREMRANGEBYSCORE', KEYS[1], 0, window_start)

-- 获取当前请求数
local request_count = redis.call('ZCARD', KEYS[1])

-- 检查是否超过阈值
if request_count >= max_requests then
    return 0 -- 限流
end

-- 添加新请求
redis.call('ZADD', KEYS[1], current_time, 'req:' .. redis.sha1(ARGV[1] .. math.random()))

-- 设置键过期时间（窗口时间+缓冲）
redis.call('EXPIRE', KEYS[1], window_seconds + 5)

return 1 -- 允许
```

3. 记录用户浏览历史

   1. 以用户id为key, 以时间戳作为 `score`，帖子作为 `member`

```redis
# 用户1001浏览帖子12345（时间戳为1698652800）
ZADD user:1001:history 1698652800 "post:12345"

# 用户1001浏览帖子67890（时间戳为1698652900）
ZADD user:1001:history 1698652900 "post:67890"

# 获取最新10条浏览记录（按时间倒序）
```

```redis
ZREVRANGE user:1001:history 0 9 WITHSCORES

# 获取最早浏览记录（按时间正序）
ZRANGE user:1001:history 0 9 WITHSCORES

# 按时间范围查询（查询指定时间段内的浏览记录）
ZRANGEBYSCORE user:1001:history 1698652800 1698653000 WITHSCORES

# 分页查询
# 第一页（每页10条）
ZREVRANGE user:1001:history 0 9 WITHSCORES

# 第二页
ZREVRANGE user:1001:history 10 19 WITHSCORES

# 删除指定时间前的记录
ZREMRANGEBYSCORE user:1001:history -inf 1698652900

# 清空浏览历史
DEL user:1001:history
```

4. 实现在线用户列表怎么做(按登录时间排序、可查询、踢人)

    1. 以用户活跃的时间戳为score, 用户id为value, 存入zset

```redis
# 用户1001在时间戳1698652800000登录/活跃
ZADD online_users 1698652800000 "user:1001"

# 获取最近活跃的前10名用户（降序）
ZREVRANGE online_users 0 9 WITHSCORES

# 获取最早活跃的前10名用户（升序）
ZRANGE online_users 0 9 WITHSCORES

```

```redis
# 获取用户1001最后活跃时间
ZSCORE online_users 1001

# 检查用户是否在线
ZSCORE online_users 1001  # 返回时间戳表示在线，nil表示离线

# 移除用户，强制下线
ZREM online_users 1001

# 清理1小时未活跃的用户（当前时间戳1698654000000）
ZREMRANGEBYSCORE online_users -inf (1698654000000-3600000)
```

bitmap

bitmap就是位图, 可以理解为是一个数组, 数组中的值只能为1或0. 它底层基于string实现.

1. 可以用来统计系统活跃用户
    1. 使用日期（精确到天）作为 key，然后用户 ID 为 offset，如果当日活跃过就设置为 1

```redis
# 2022年3月22活跃用户统计，id为111的用户活跃，id为222的用户活跃
setbit 20220322 111 1
setbit 20220322 222 1

# 2022年3月23活跃用户统计，id为111的用户活跃
setbit 20220323 111 1

# 统计2022年3月22的活跃用户
bitcount 20220322
```

2. 可以统计用户签到
    1. 以年月拼接用户做为key，如`user_sign:{年}:{月}:{用户ID}`，用户天签到一次，就把对应位置-1 置为1
        1. 如当用户在2025年6月1日签到1次，就把bitmap位置0置为1

```redis
# 用户123456在2025年6月1日，签到1次
SETBIT user_sign:2025:6:123456 0 1

# 用户123456在2025年6月2日，签到1次
SETBIT user_sign:2025:6:123456 1 1

# 用户123456在2025年6月13日，签到1次
SETBIT user_sign:2025:6:123456 12 1

# 检查用户123456在2025年6月13日是否签到
GETBIT user_sign:2025:6:123456 12

# 统计用户123456在2025.6的签到天数
BITCOUNT user_sign:2025:6:123456
```

3. 补签的实现思路

1. 再创建一个用来记录是否补签的bitmap, 如 `user_complement_sign:{年}:{月}:{用户ID}`
2. 如果某天的补签是补签的, 则将这个新的bitmap对应的位置标1, 不是补签则为0.
3. 通过看之前的bitmap可以看到是否签到, 通过看新的bitmap, 可以看到是否为补签.

hyperloglog

HyperLogLog可以用极小的内存估算海量数据去重计数, 但是会有一点误差.

1. HyperLogLog可以做页面的uv统计
    1. 以页面路径为key, 以用户id为value放入HyperLogLog

```redis
# 将2025年6月1日, 访问page1页面的用户ID (用户分别为111 112 113) 添加到 HyperLogLog 中
pfadd page1:20250601:uv 111 112 113

# 统计2025年6月1日, 访问page1页面的用户, 不同统计重复插入的值(同一个用户重复访问不会统计)
pfcount page1:20250601:uv

# 合并2025年6月1日, 2025年6月2日, 2025年6月3日对应的HLL (每天统计, 然后合并一个月的, 就能计算出一个月的uv总量)
pfmerge page1:20250601:uv page1:20250602:uv page1:20250603:uv
```